Course: Computer Vision

Course Code: ITA0506

Slot: A

Faculty: Jeni Gracia R J

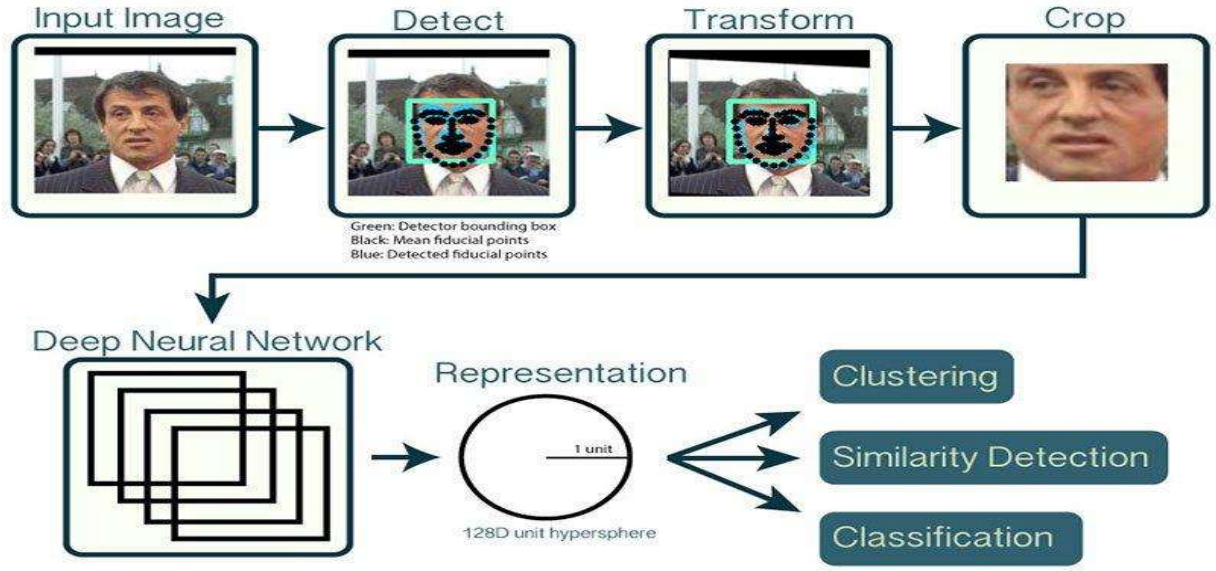
Submitted By

Y Sravan Kumar (192472289)

CO5 – ASSESSMENT 1

CONSTRAINT-BASED PROBLEM SOLVING

1. REAL-TIME FACE RECOGNITION FOR CAMPUS SECURITY



Problem

A university wants to identify registered students and staff using CCTV cameras at the entrance. The system must work in real time, even under different lighting conditions, while minimizing false recognition.

Proposed OpenCV Pipeline

CCTV / Camera
↓
Image Acquisition
↓
Preprocessing
↓
Face Detection
↓
Face Alignment
↓
Feature Extraction
↓
Feature Matching
↓
Registered Face Database
↓
Identity + Confidence Score
↓
Display / Security Alert

1. Image Acquisition

OpenCV's VideoCapture() can be used to continuously capture frames from the CCTV camera.

Camera → Video Frames → OpenCV

The system processes the incoming frames continuously.

2. Preprocessing

The captured image is improved before detecting the face.

Important techniques:

Resize the frame.

Convert image to grayscale when appropriate.

Apply histogram equalization or CLAHE for uneven lighting.

Apply Gaussian or median filtering to reduce noise.

Normalize the face image.

Why?

Preprocessing improves image quality and makes face detection more reliable under different lighting conditions.

3. Face Detection

A face detector identifies the location of faces in the frame.

Possible OpenCV techniques:

Haar Cascade

OpenCV DNN face detector

YuNet face detector

The detector produces bounding boxes:

(x, y, width, height)

around detected faces.

4. Face Alignment

The detected face can be aligned based on important facial landmarks such as:

Eyes

Nose

Mouth

This reduces the effect of head position and slight rotation.

5. Feature Extraction

The system converts the face into numerical features that can be compared with registered faces.

Suitable methods include:

LBPH

Eigenfaces

Fisherfaces

Deep face embeddings

For a low-cost computer, a lightweight feature extraction approach can be selected to reduce computational requirements.

6. Recognition

The extracted features are compared with the registered face database.

Example:

Input Face
↓
Extract Features
↓
Compare with Database
↓
Closest Match
↓

Student ID: 1023

Confidence: 92%

A confidence/distance threshold should be used.

If the similarity is insufficient:

Unknown Person

instead of incorrectly assigning an identity.

7. Minimizing False Recognition

False recognition can be reduced by:

Using a strict matching threshold.

Registering multiple images for each person.

Using different facial poses during registration.

Applying face alignment.

Improving illumination.

Rejecting low-quality face images.

Requiring consistent recognition across multiple frames.

For example, instead of accepting one frame:

Frame 1 → John

Frame 2 → John

Frame 3 → John

↓

Confirm John

This is more reliable than identifying a person from only one frame.

8. Real-Time Optimization

The system must operate on a low-cost computer.

Important optimization techniques:

Resize frames

Process smaller frames instead of full-resolution CCTV frames.

Region of Interest

Only process the entrance area instead of the entire camera frame.

Frame skipping

For example:

Frame 1 → Process
Frame 2 → Skip
Frame 3 → Skip
Frame 4 → Process

Lightweight model

Use lightweight face detection and feature extraction models.

Multi-threading

Camera capture and processing can be performed separately.

9. Accuracy and Performance

Accuracy is improved through:

Good-quality registration images.

Lighting normalization.

Face alignment.

Feature extraction.

Matching thresholds.

Multi-frame confirmation.

Real-time performance is improved through:

Frame resizing.

ROI processing.

Frame skipping.

Lightweight models.

Efficient OpenCV functions.

Expected Output

Face Detected
Name : Registered Student
Student ID : 1023
Confidence : 92%
Status : Authorized

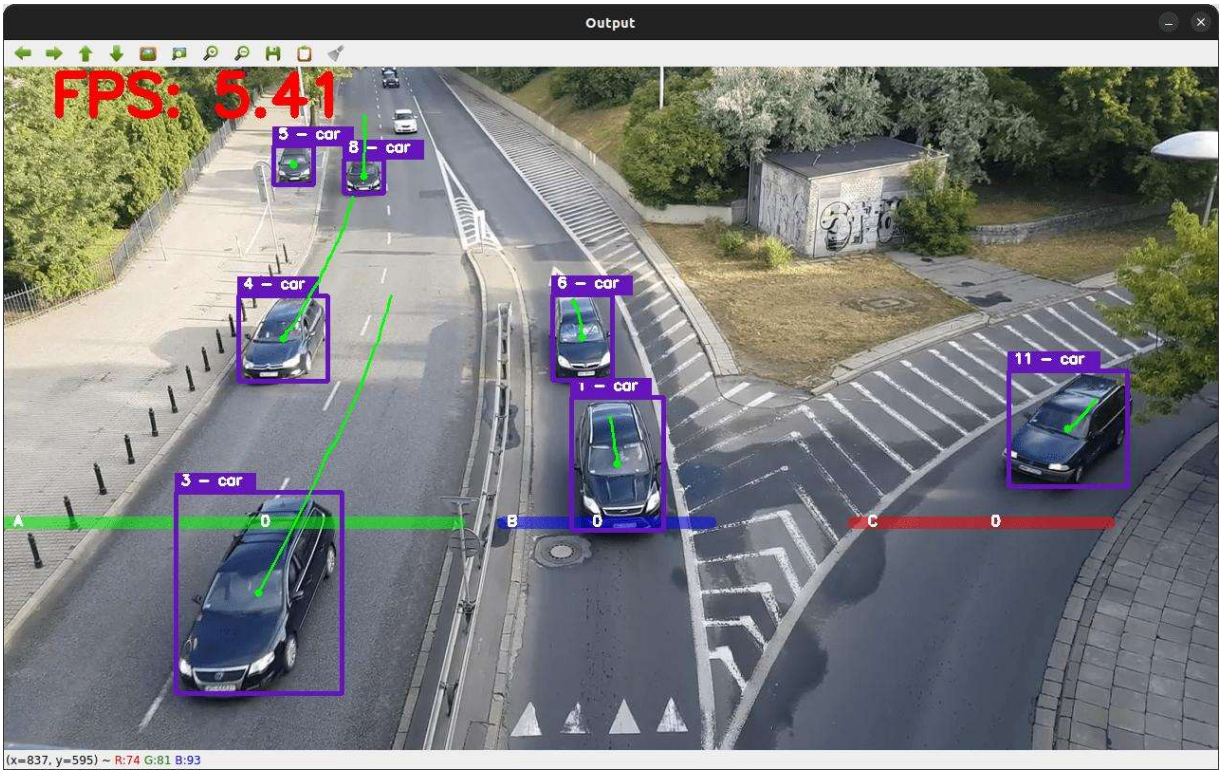
If there is no reliable match:

Name : Unknown
Confidence : Low
Status : Alert

Conclusion

The proposed OpenCV pipeline captures faces from CCTV, preprocesses the images, detects faces, extracts facial features, compares them with registered data, and identifies individuals. Thresholding, multi-frame confirmation, preprocessing, ROI processing, and lightweight models help achieve accuracy and real-time performance on low-cost hardware.

2. INTELLIGENT TRAFFIC VIOLATION DETECTION SYSTEM


Output
FPS: 5.41
5 – car
8 – car
4 – car
6 – car
1 – car
11 – car
3 – car
A
0
B
0
C
0
(x=837, y=595) ~ R:74 G:81 B:93


Input
Blob Tracking
PBAS
VehicleCouting
HORIZONTAL (A) (B)
STATE: B
A->B: 5

Problem

A traffic department wants to automatically monitor vehicles using roadside cameras. The system must detect, track, count vehicles, identify line crossing, and detect abnormal movements.

Proposed OpenCV Pipeline

Roadside Camera
↓
Video Acquisition
↓
Preprocessing
↓
Vehicle Detection
↓
Object Tracking
↓
Line-Crossing Detection
↓
Vehicle Counting
↓
Violation Detection
↓
Visualization + Alert

1. Video Acquisition

OpenCV VideoCapture() is used to capture the live traffic video.

Camera
↓
Video Frames
↓
OpenCV

2. Preprocessing

Before vehicle detection, frames can be processed using:

Resizing

Gaussian blur

Median filtering

Grayscale conversion

Noise reduction

The objective is to reduce unnecessary noise while preserving vehicle information.

3. Vehicle Detection

Vehicles can be detected using object detection techniques such as:

Haar Cascade for simple applications.

HOG-based detection.

YOLO.

SSD.

MobileNet-SSD.

A modern lightweight detector such as YOLO with a small model or MobileNet-SSD can provide better real-time performance.

Detected objects can include:

Car
Bus
Truck
Motorcycle

4. Object Tracking

Detection alone identifies vehicles in individual frames. Tracking maintains the identity of a vehicle across multiple frames.

Possible tracking techniques include:

Centroid tracking

KCF

CSRT

Kalman Filter

SORT/Deep SORT

Example:

Frame	1	→	Vehicle	ID	01
Frame	2	→	Vehicle	ID	01
Frame	3	→	Vehicle	ID	01

Frame 4 → Vehicle ID 01

Therefore, the same vehicle is not counted repeatedly.

5. Line-Crossing Detection

A virtual line is placed on the road.

---------- ← Counting Line

🚗

↓

🚗

↓

---------- ← Vehicle crossed

The system compares the vehicle's position before and after crossing the line.

If:

Previous position = Above line

Current position = Below line

then the vehicle is counted.

6. Vehicle Counting

Each tracked vehicle receives a unique ID.

For example:

Vehicle ID 1 → Crossed

Vehicle ID 2 → Crossed

Vehicle ID 3 → Crossed

Count:

Total Vehicles = 3

This prevents the same vehicle from being counted multiple times.

7. Abnormal Movement Detection

The system can detect:

Wrong-way movement.

Sudden direction changes.

Stopped vehicles.

Vehicles entering restricted areas.

Lane violations.

Unusual movement patterns.

For example:

Expected direction → → →

Vehicle movement ← ← ←

↓

Wrong-Way Alert

8**. Handling Shadows**

Shadows can sometimes be detected as part of vehicles.

To reduce this problem:

Use suitable color-space processing.

Apply morphological operations.

Use object detectors rather than relying only on background subtraction.

Use shape and size constraints.

9. Handling Camera Noise

Camera noise can be reduced using:

Gaussian Blur
Median Filter

These techniques smooth unwanted pixel-level **variations.**

10. Handling Dynamic Backgrounds

Trees, moving leaves, rain, and changing illumination can affect background subtraction.

Solutions include:

Adaptive background models.

MOG2 background subtraction.

KNN background subtraction.

Object detection combined with motion analysis.

11. Handling Multiple Vehicles

Each detected vehicle should have a unique tracking ID.

Car → ID 01

Bus → ID 02

Truck → ID 03

Car → ID 04

This prevents duplicate counting.

12. Real-Time Optimization

To achieve real-time performance:

Resize frames.

Process selected regions.

Use lightweight object detectors.

Skip unnecessary frames.

Use GPU acceleration if available.

Use efficient tracking algorithms.

Expected Output

Vehicle Count : 37

Vehicle ID 12 → Normal

Vehicle ID 13 → Normal

Vehicle ID 14 → Wrong Way

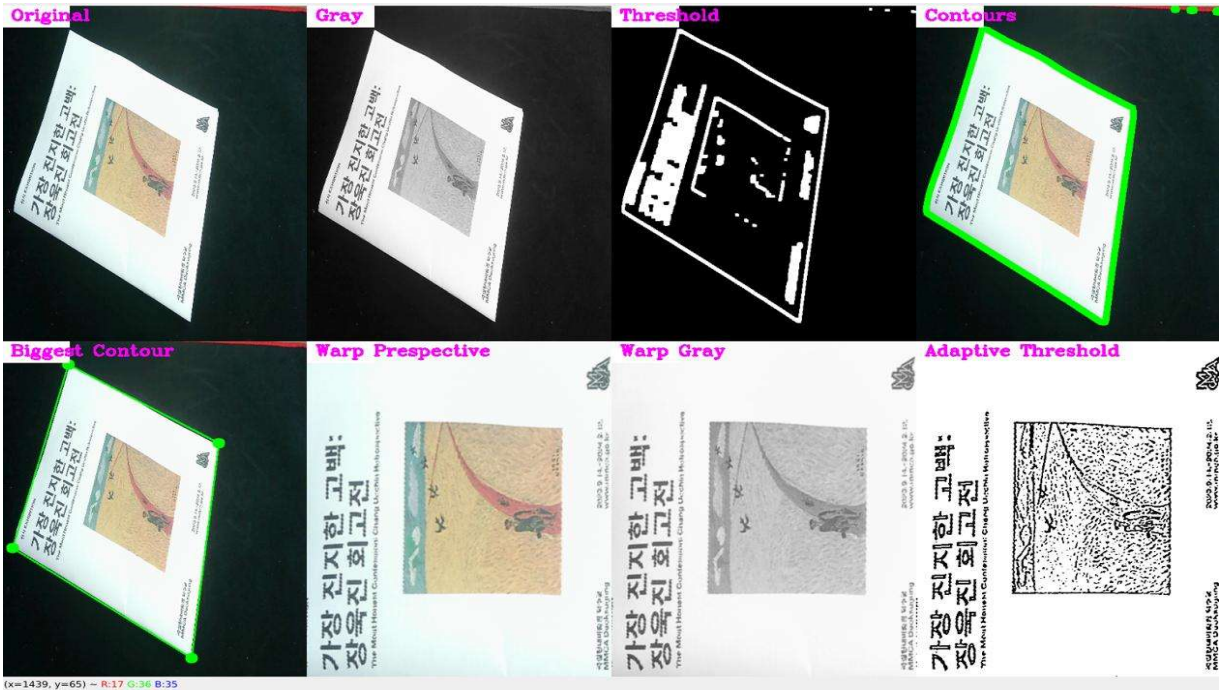
ALERT: Traffic Violation Detected

Conclusion

The system combines video acquisition, preprocessing, object detection, tracking, line-crossing analysis, motion analysis, and visualization. Unique vehicle IDs prevent duplicate counting, while

filtering, adaptive background models, and lightweight detectors reduce errors and maintain real-time performance.

3. AUTOMATED DOCUMENT PROCESSING SYSTEM



Problem

A company receives thousands of scanned documents containing:

Blurred text.

Uneven illumination.

Noise.

Different font sizes.

Unwanted backgrounds.

The system must automatically improve the documents before OCR.

Proposed Pipeline

Scanned Document

↓

Image Acquisition

↓

Grayscale Conversion

↓

Noise Removal

↓

Illumination Correction

↓

Thresholding

↓

Morphological Operations

↓

Geometric Correction

↓

Text Segmentation

↓

OCR

↓

Readable Text

1. Image Acquisition

The scanned document is loaded using OpenCV.

Document Image → OpenCV → Processing

2. Grayscale Conversion

A color document is converted into grayscale.

Color Image

↓

Grayscale Image

Purpose

It reduces three color channels into one intensity channel, making further processing faster and simpler.

3. Noise Removal

Noise can be reduced using:

Gaussian Blur

Median Blur

Bilateral Filtering

For document images, median filtering can be useful for removing certain types of noise while preserving edges.

4. Thresholding

Thresholding separates text from the background.

Common methods:

Binary thresholding.

Otsu thresholding.

Adaptive thresholding.

For uneven illumination, adaptive thresholding can be particularly useful because different regions can use different thresholds.

5. Morphological Operations

Morphological operations improve text regions.

Opening

Useful for removing small unwanted noise.

Opening = Erosion → Dilation

Closing

Useful for connecting small gaps in characters.

Closing = Dilation → Erosion

6. Geometric Correction

Scanned documents may be rotated or tilted.

The system can detect document boundaries and correct the perspective.

Tilted Document

↓

Corner Detection

↓

Perspective Transformation

↓

Straight Document

This is also called deskewing/perspective correction depending on the distortion.

7. Text Segmentation

The document is divided into meaningful regions:

Paragraphs.

Lines.

Words.

Text blocks.

Contours and connected components can help identify text regions.

8. OCR

After preprocessing, the cleaned image is given to an OCR engine such as Tesseract.

Processed Image

↓

OCR

↓

Extracted Text

9. Batch Processing

Since thousands of documents need to be processed, the system can process files automatically:

Document 1 → Process → OCR

Document 2 → Process → OCR

Document 3 → Process → OCR

...

Document N → Process → OCR

10. How Each Stage Improves OCR

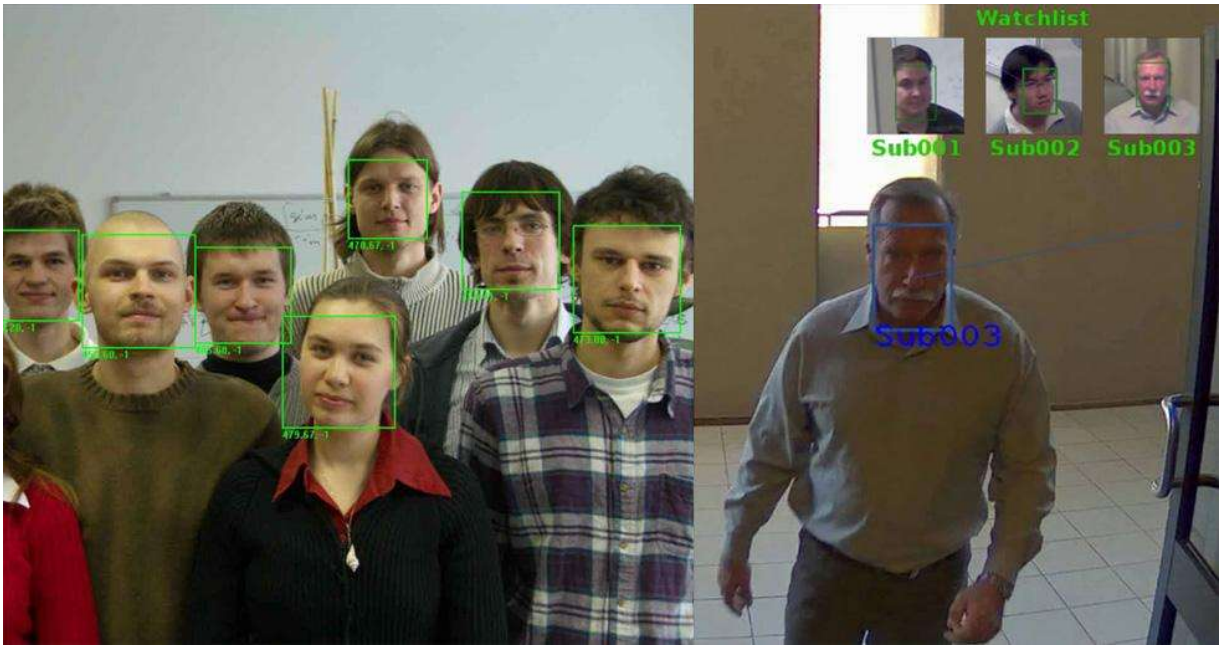
Technique	Purpose
Grayscale	Simplifies image processing
Filtering	Removes noise
Thresholding	Separates text from background
Morphology	Improves character regions
Segmentation	Separates text regions

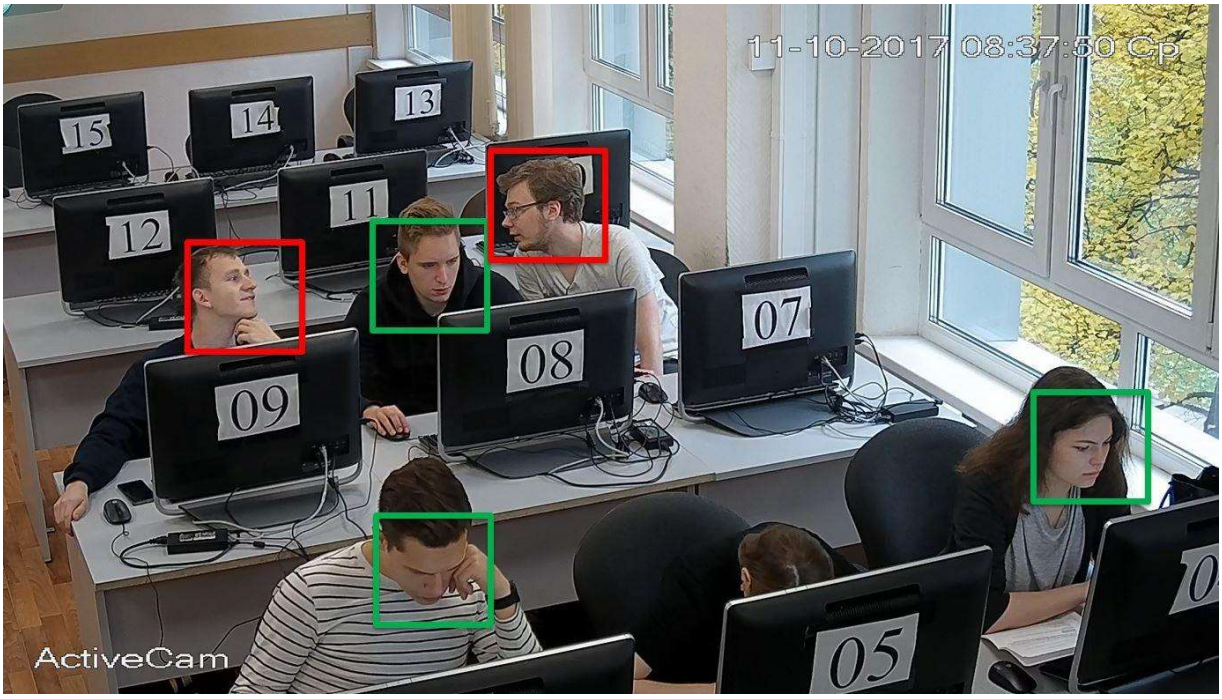
Deskewing	Corrects tilted documents
Perspective correction	Corrects document geometry
OCR	Converts image text into digital text

Conclusion

The proposed OpenCV pipeline improves OCR accuracy by removing noise, correcting illumination, separating text from the background, improving character regions, and correcting document geometry before OCR.

4. REAL-TIME MULTI-PERSON TRACKING AND ATTENDANCE SYSTEM





Problem

A college wants to automatically record attendance using classroom CCTV cameras. The main challenge is that the same student may appear in many consecutive frames.

Therefore, the system must detect, recognize, track, and avoid duplicate attendance.

Proposed Pipeline

Classroom CCTV

↓

Frame Acquisition

↓

Preprocessing

↓

Multi-Face Detection

↓

Face Alignment

↓

Feature Extraction

↓

Face Recognition

↓

Person Tracking

↓

Identity Management

↓

Attendance Database

↓

Attendance Report

1. Face Detection

Multiple faces are detected in each frame.

For example:

Student	A	→	Face	1
Student	B	→	Face	2
Student	C	→	Face	3

Student D → Face 4

2. Feature Extraction

Features are extracted from each detected face.

Possible methods:

LBPH.

Eigenfaces.

Fisherfaces.

Deep face embeddings.

The extracted representation is compared with registered student data.

3**. Face Recognition**

The system compares the detected face with the registered database.

Example:

Face → Feature Extraction → Database Matching

Face 1 → Student 101

Face 2 → Student 102

Face 3 → Student 103

4. Tracking

Recognition does not need to happen on every frame.

Once a student is recognized, the tracking system maintains the identity.

Frame 1 → Student 101

Frame 2 → Track ID 5

Frame 3 → Track ID 5

Frame 4 → Track ID 5

This reduces processing requirements.

5. Maintaining Identity

Each person can be assigned a tracking ID.

Track ID 1 → Student 101

Track ID 2 → Student 102

Track ID 3 → Student 103

The tracking algorithm follows the person even when the face is temporarily not detected.

6. Avoiding Duplicate Attendance

A database or set can maintain the students already marked present.

Example:

Present = {101, 102, 105}

If Student 101 is recognized again:

Student 101 → Already Present → Do not mark again

Therefore, each student gets attendance only once.

7. Handling Temporary Face Disappearance

A face may disappear because of:

Head movement.

Occlusion.

Student turning away.

Temporary obstruction.

The tracker can maintain the identity for a limited number of frames.

Face detected
↓
Track person
↓
Face temporarily disappears
↓
Continue tracking
↓
Face appears again
↓
Confirm identity

8. Reducing False Attendance

False attendance can be reduced using:

Recognition confidence threshold.

Multi-frame confirmation.

High-quality registered images.

Face alignment.

Tracking consistency.

Duplicate checking.

9. Real-Time Optimization

Use:

Lower-resolution frames.

ROI.

Lightweight face detector.

Frame skipping.

Tracking instead of repeated recognition.

GPU acceleration when available.

Expected Output

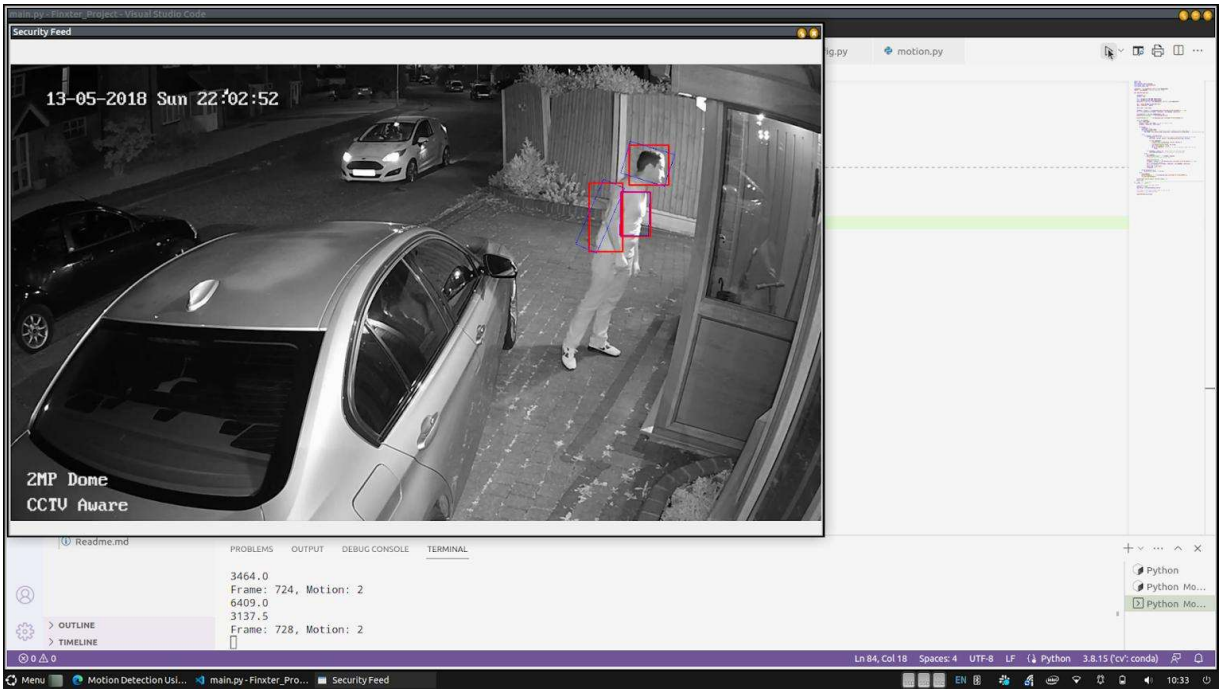
Student ID	Name	Attendance
101	Rahul	Present
102	Priya	Present
103	Arjun	Present
104	Sneha	Present

Conclusion

The system combines face detection, feature extraction, recognition, tracking, and attendance management. Tracking maintains identity between frames, while a database prevents duplicate attendance. Frame skipping and lightweight models help maintain real-time performance.

5. SMART HOME SURVEILLANCE AND INTRUSION DETECTION


Security Feed
Room Status: Occupied
Tuesday 19 May 2015 12:25:07PM


Security Feed
13-05-2018 Sun 22:02:52
2MP Dome
CCTV Aware
motion.py
Readme.md
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
3464.0
Frame: 724, Motion: 2
6409.0
3137.5
Frame: 728, Motion: 2
OUTLINE
TIMELINE
Python
Python Mo...
Python Mo...
Ln 84, Col 18
Spaces: 4
UTF-8
LF
Python
3.8.15 ('cv': conda)
Menu
Motion Detection Usi...
main.py - Finxter_Pro...
Security Feed
10:33

Problem

A fixed camera continuously monitors a home entrance. The system should detect meaningful movement and generate an alert when an unauthorized person enters a predefined region.

The major challenge is distinguishing real movement from shadows, trees, lighting changes, and camera noise.

Proposed Pipeline

Security Camera
↓
Video Acquisition
↓
Preprocessing
↓
Background Subtraction
↓
Noise Filtering
↓
Morphological Operations
↓
Contour Detection
↓
Object Detection
↓
Object Tracking
↓
Restricted Region Check
↓
Intrusion Alert

1. Video Acquisition

OpenCV captures frames continuously from the fixed security camera.

Camera → Video Frames → OpenCV

2. Preprocessing

The captured frame can be processed using:

Grayscale conversion.

Gaussian blur.

Median filtering.

Frame resizing.

The purpose is to reduce unwanted noise.

3. Background Subtraction

Since the camera is fixed, the background can be modeled.

When an object moves:

Current Frame - Background

↓

Difference

↓

Moving Object

OpenCV methods such as:

MOG2

KNN

can be used for background subtraction.

4. Morphological Operations

Background subtraction can produce small noise regions.

Morphological operations help clean the detected foreground.

Opening

Removes small noise.

Closing

Fills small gaps in detected objects.

5. Contour Detection

Contours are extracted from the foreground mask.

Very small contours can be ignored using an area threshold.

For example:

```
if contour_area > minimum_area:
    consider as moving object
```

This prevents tiny movements from triggering alerts.

6. Motion Analysis

The system calculates:

Object position.

Movement direction.

Object size.

Speed.

Movement duration.

This helps distinguish meaningful movement from small background changes.

7. Tracking

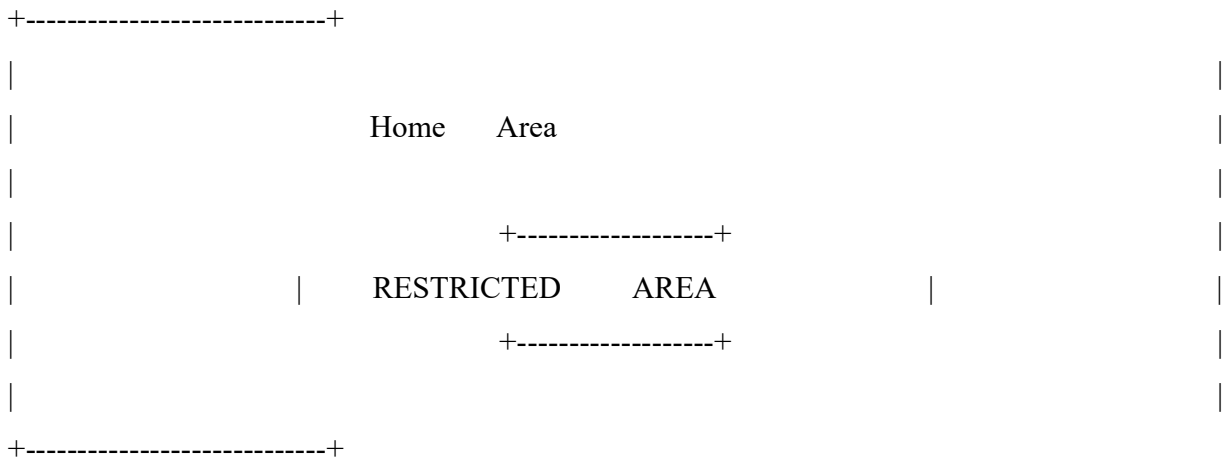
Once a moving object is detected, it is assigned a tracking ID.

Object → Track ID 01

The tracker follows the object across consecutive frames.

8. Predefined Region

A restricted region can be defined around the entrance.



If an unauthorized object enters this region, the system generates an alert.

9. Reducing False Alarms

Shadows

Shadows can be reduced using:

Color-space analysis.

Morphological filtering.

Object-size constraints.

Illumination Changes

Sudden lighting changes can affect background subtraction.

Adaptive background models can help update the background.

Trees and Moving Leaves

Small moving regions can be ignored using contour-area thresholds.

Camera Noise

Gaussian or median filtering can reduce small pixel-level noise.

10. Suspicious Activity Detection

The system can generate alerts for:

Person entering restricted region.

Long-duration movement near entrance.

Unusual movement direction.

Movement during restricted hours.

Example:

Person Detected

↓

Restricted Region?

↓

YES

↓

Intrusion Alert

11. Real-Time Optimization

For low computational resources:

Resize frames.

Process only the entrance ROI.

Use lightweight background subtraction.

Skip unnecessary frames.

Track objects instead of detecting everything repeatedly.

Use efficient OpenCV operations.

Expected Output

Motion Detected

Object ID : 04

Object : Person

Location : Entrance

Status : Unauthorized Entry

!!! INTRUSION ALERT !!!

Conclusion

The proposed surveillance system uses background subtraction, filtering, morphological operations, contour detection, motion analysis, and tracking to identify suspicious activity. Noise filtering, area thresholds, adaptive background models, and restricted-region detection reduce false alarms while allowing real-time operation.